Software Development
and
Professional Practice

Process Life Cycle Models

By

Dr. Rasha Mahmoud Kamel

كلية
الحاسبات والمعلومات

جامعة
أسيوط

Introduction

- The process of developing software is commonly referred to as the Software Development Lifecycle (**SDLC**).
- Every software system, regardless of its size, has a life cycle that is broadly composed of the following steps:
  - **(1)** Conception.
  - **(2)** Requirements analysis.
  - **(3)** Design.
  - **(4)** Coding and debugging.
  - **(5)** Testing.
  - **(6)** Release.
  - **(7)** Maintenance/software evolution.
- The development process may combine multiple steps or iterate over a subset of steps repeatedly between releases.

Introduction

- However, in one form or another, all development processes should encompass all the life cycle steps to produce high-quality software.
- The management of software development projects generally falls into two main types:
  - Traditional ***plan-driven*** models
  - Newer ***agile development*** models

Plan-driven Models

- In plan-driven models, the project team will generally ***do a complete life cycle*** before going back in the SDLC to start working on the next version of the product.
- The ***process*** is ***generally stricter*** in terms of defined steps and when releases happen.
- These models have ***more clearly defined phases***, with ***formal sign-off required*** at the completion of each phase before moving on to the next phase.
- They also ***require extensive documentation at each phase*** and verification of completion of each work product.
- Plan-driven approaches tend to ***work well for government contracts*** for new software ***with well-defined deliverables***.

Agile Models

- In agile development, which is more prevalent today, ***the project team*** typically ***completes a partial life cycle***—usually steps 3 through 5—and iterate through these steps multiple times before proceeding to the release step.
- Agile models are inherently ***incremental*** and operate under the assumption that ***small, frequent releases*** produce a more robust product than ***larger, less frequent ones***.
- ***Phases*** in agile models tend to ***blur together*** more than in plan-driven models.
- Agile approaches also require ***less formal documentation at each phase*** compared with plan-driven models, as the basic idea of agile models being that code is what is being produced and so documentation efforts should focus there.

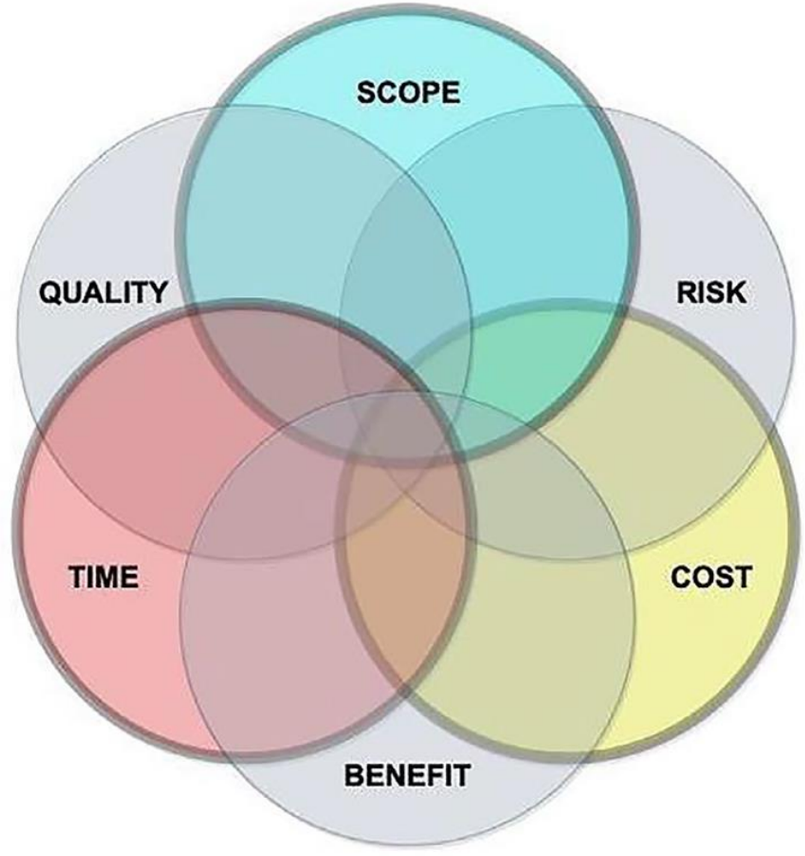
Software Development Process

- There is ***no single best process*** for developing software.
- Each project must select the model that ***best suits its particular software*** where this decision based on the
  - Project domain.
  - Size of the project.
  - Experience of the team.
  - Timeline of the project.

Variables of Software Development

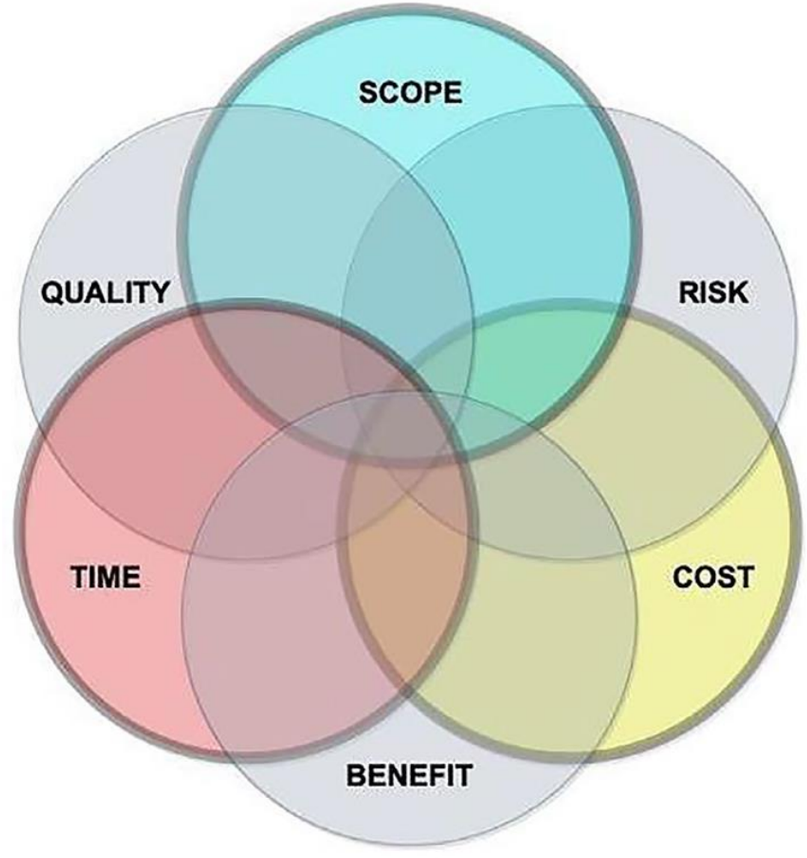
- The three main variables of software development are:

  (1) **Cost:** This refers to ***the budget and resources allocated for development***. It includes salaries of developers, tools, infrastructure, and operational expenses. Cost influences factors such as the team size and types of tools available during development. For small companies and startups, cost may also affect the working environment. It is usually the most constrained variable, and developers typically have limited control over it.



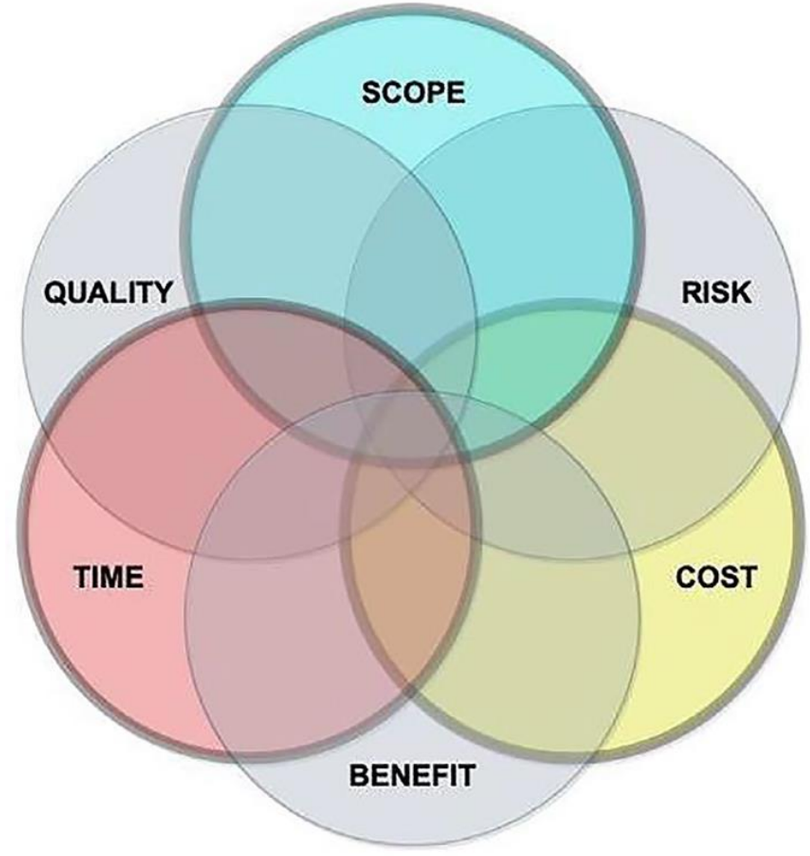
Variables of Software Development

(2) Time: This refers to ***the schedule or deadline for delivering the product***. It is usually imposed on developers from the outside. For example, many consumer products must be released in late summer or early fall to hit the holiday buying season. If a project falls behind schedule, the only way to fix the problem is to drop features or lowering quality, neither of which is desirable.



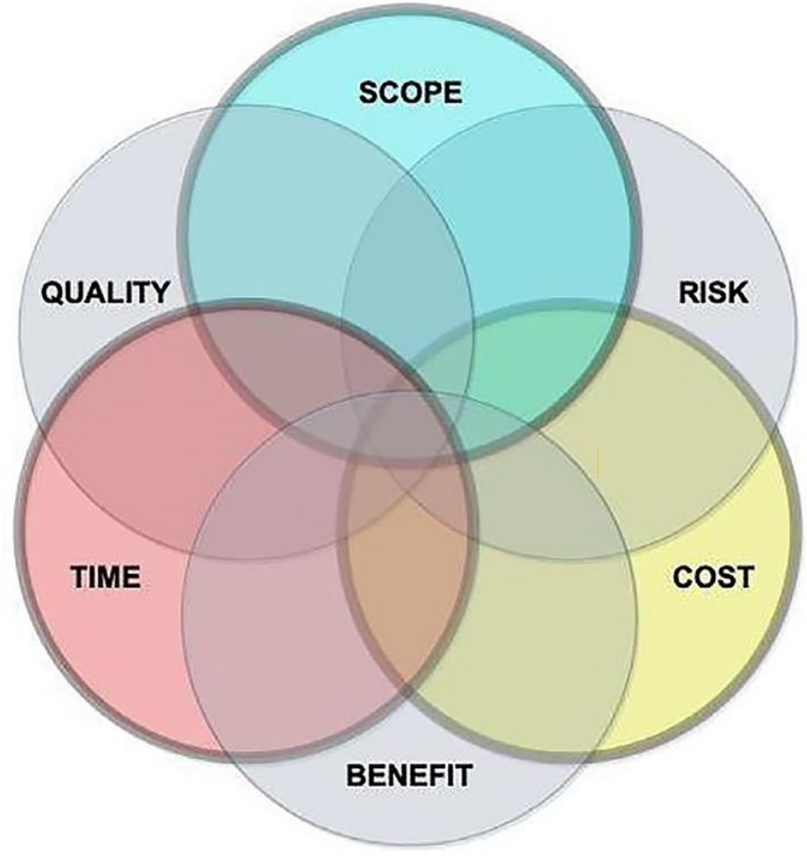
Variables of Software Development

(3) Scope (Features): It defines ***what features will be included in the project*** (what the product actually does). This is what developers should always focus on. It is the most important of the variables from the customer's perspective and it is also the one you as a developer have the most control over. If the developers don't have control over the feature set for each release, new features might keep getting added (scope creep), making it difficult to complete the work on time. This leads to missed deadlines. This is why developers should do the estimates for software work products.



Variables of Software Development

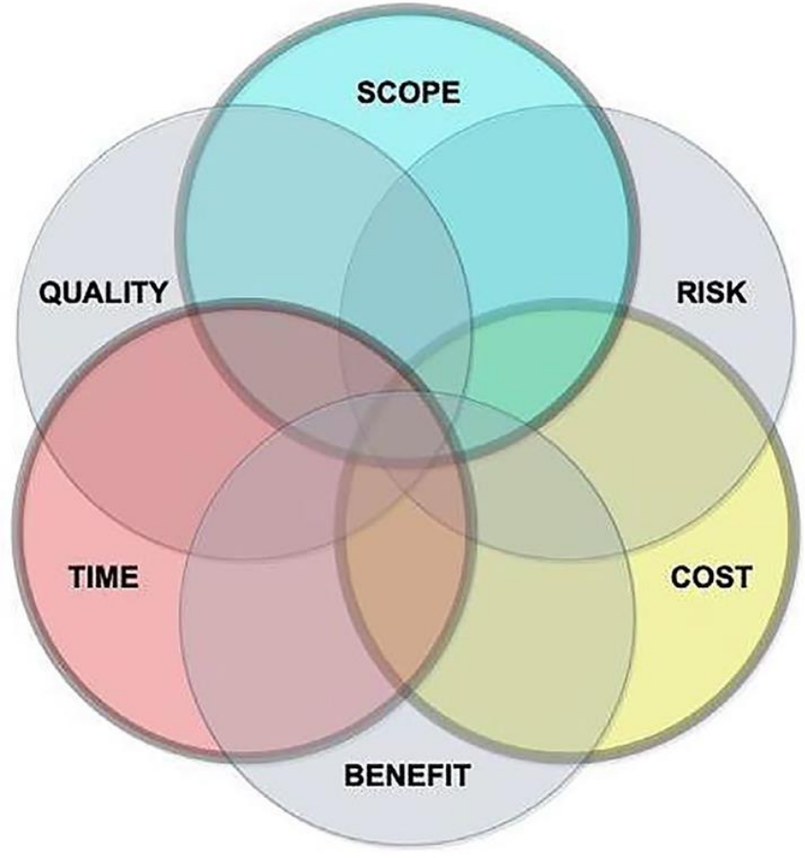
- Together, cost, time, and scope form the Iron Triangle of software development, in which specifying any two of the variables defines the third. For example:
  - If time and cost are fixed, then scope must adjust.
  - If scope and time are fixed, then cost must adjust.
  - If scope and cost are fixed, then time must adjust.
- In practice, customers often specify both the timeframe and budget. These become the primary constraints, and the development team must then estimate a realistic scope that can be delivered within those limits.



Variables of Software Development

- A fourth variable—quality—is added to the variables.

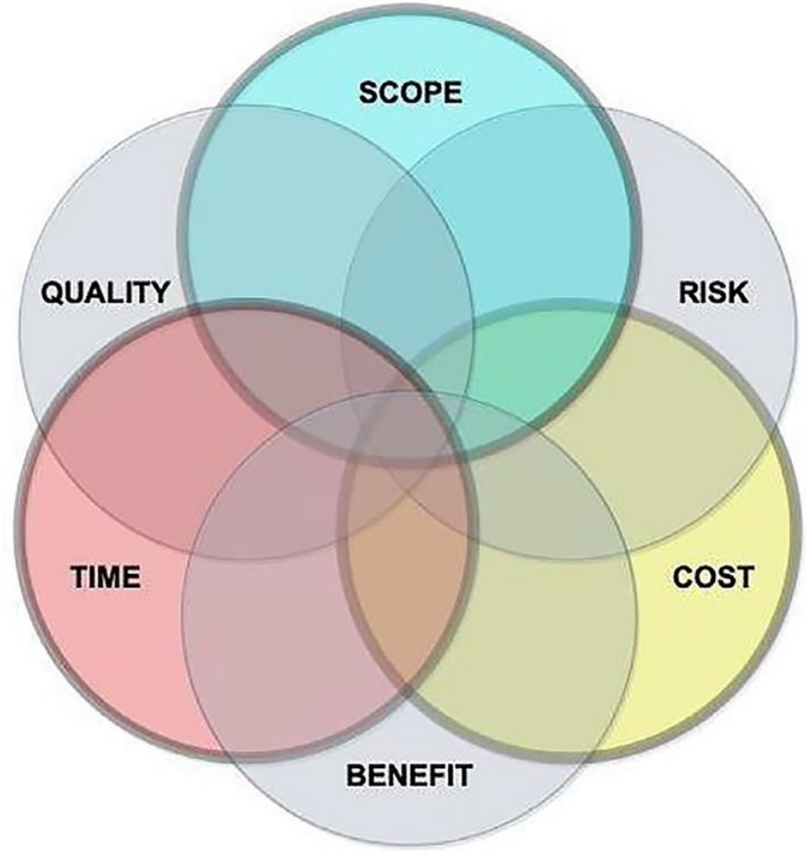
(4) Quality: This refers to how well the software performs. It includes ***the level of reliability, maintainability, and performance***. It can also be measured by the number/severity of defects or functionality reductions you are willing to release with. Higher quality may increase time and cost but reduces defects and rework. You can make short-term gains in delivery schedules by sacrificing quality, but the cost is enormous. It will take more time to fix the next release and your credibility will be significantly damaged.



Variables of Software Development

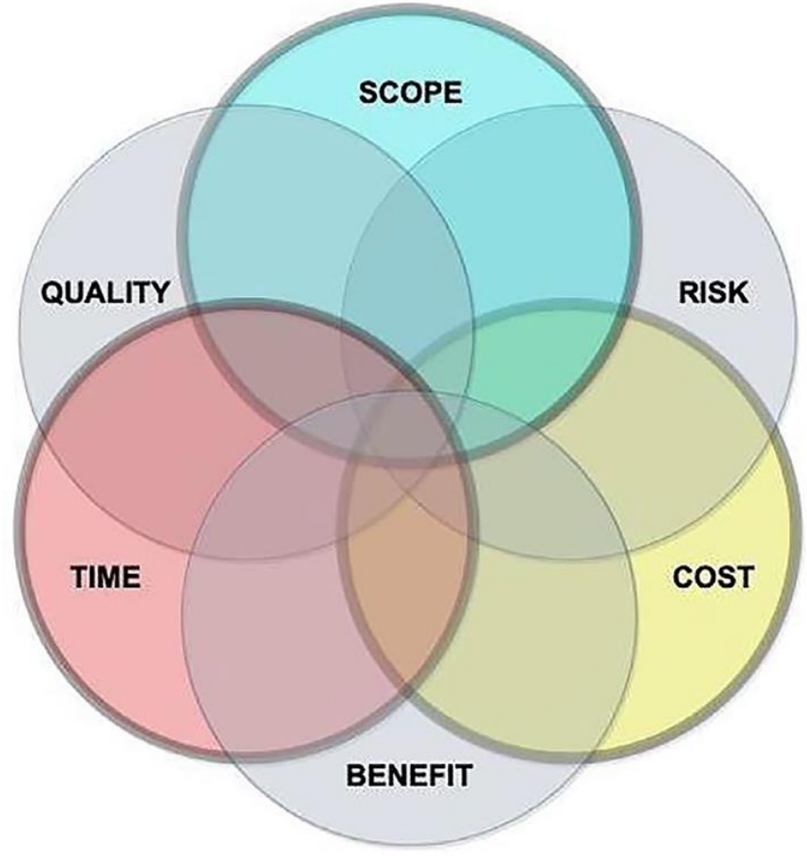
- Recently, two additional variables have been introduced as part of an expanded six-variable set of project constraints:

  (5) Benefit: This refers to the ***value delivered to both the product provider and the client***. Changes in the market conditions, client needs, or the target audience, may change the expected benefits, which in turn may require changes to scope or other project variables.



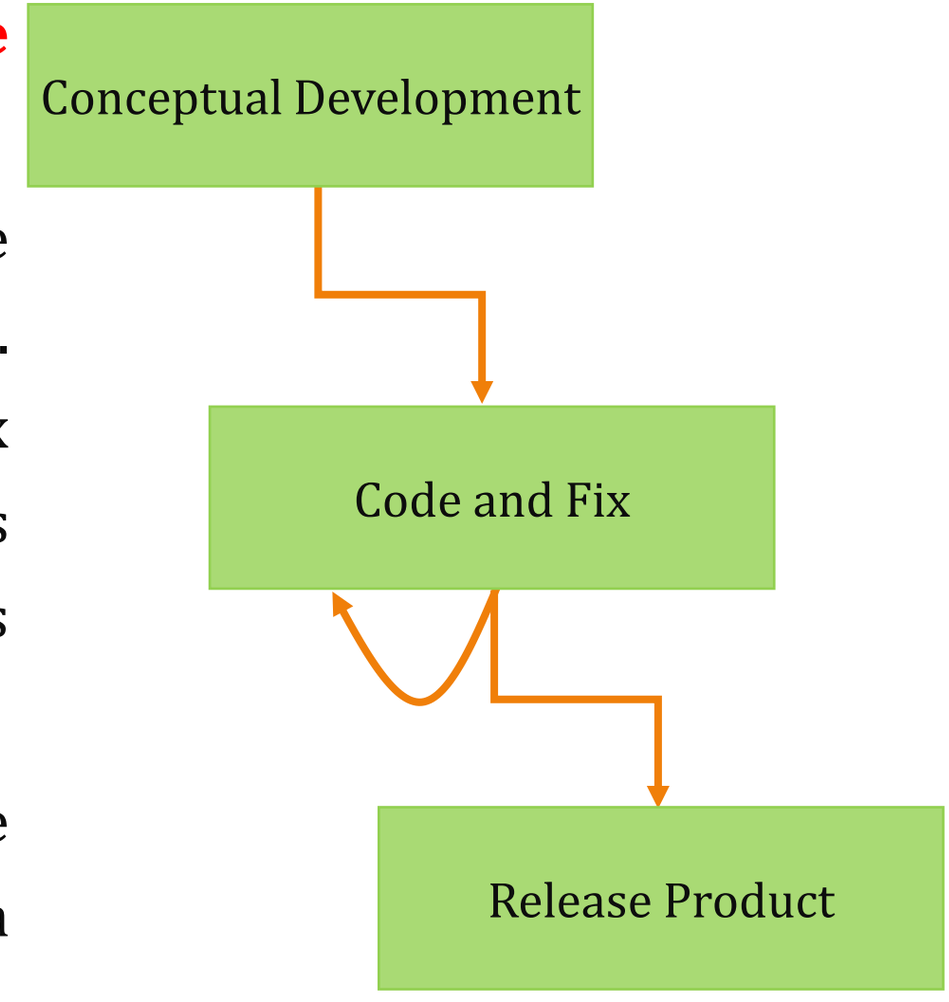
Variables of Software Development

(6) Risk: This represents the ***likelihood of failing to achieve the intended values for the other project variables***, such as failing to implement all planned features, increasing development time, or exceeding the expected budget. Managing one type of risk may increase another (e.g., extending the schedule may reduce the scope risk, but is likely to increase cost). To minimize overall risk, developers must control as many of the variables as possible. By anticipating change and effectively coping with change as it occurs, developers can keep the risk and cost of change under control.



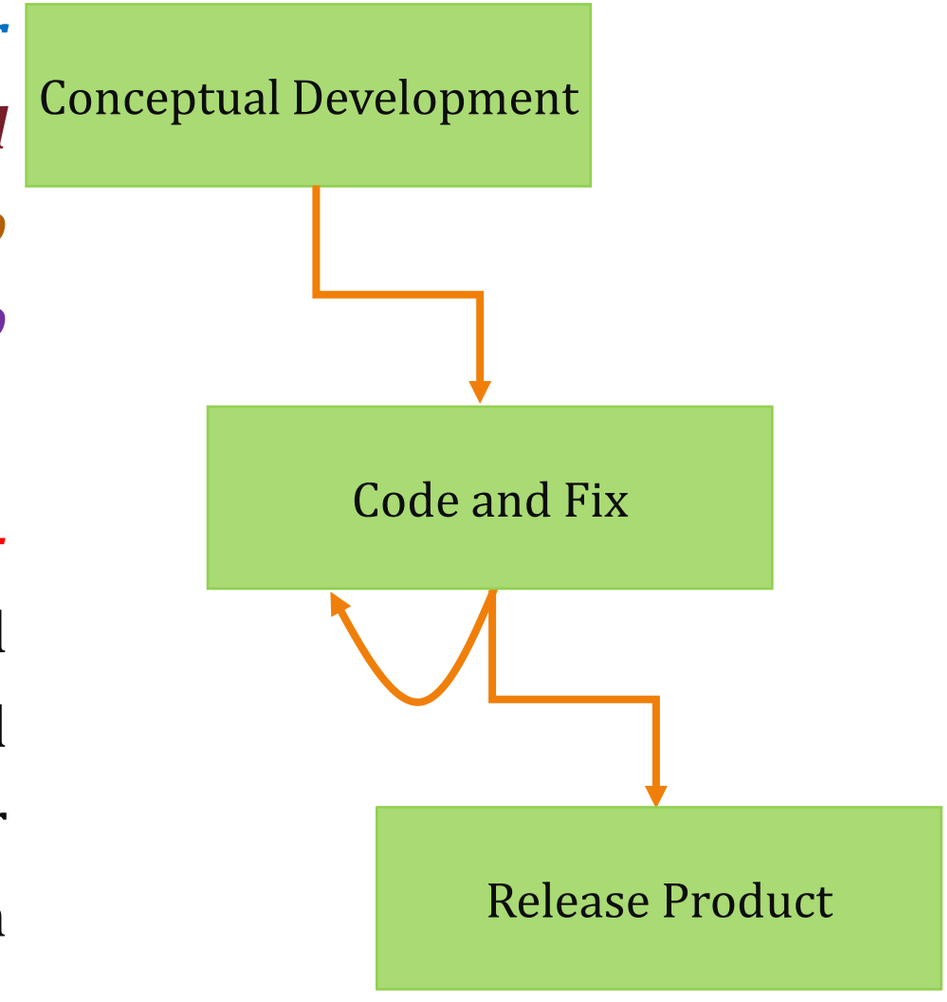
Code and Fix Model

- The first model of software development is the **code and fix model**. ***It isn't really a model*** at all.
- In this model, you just take a minimal amount of time to understand the problem and then start coding. Compile the code and try it out. If it doesn't work, fix the first problem you see and try it again. Continue this cycle of type-compile-run-fix until the program does what you want with no fatal errors and then ship it.
- It is what most of developers do when they are working on small projects, either individually or with a single partner.



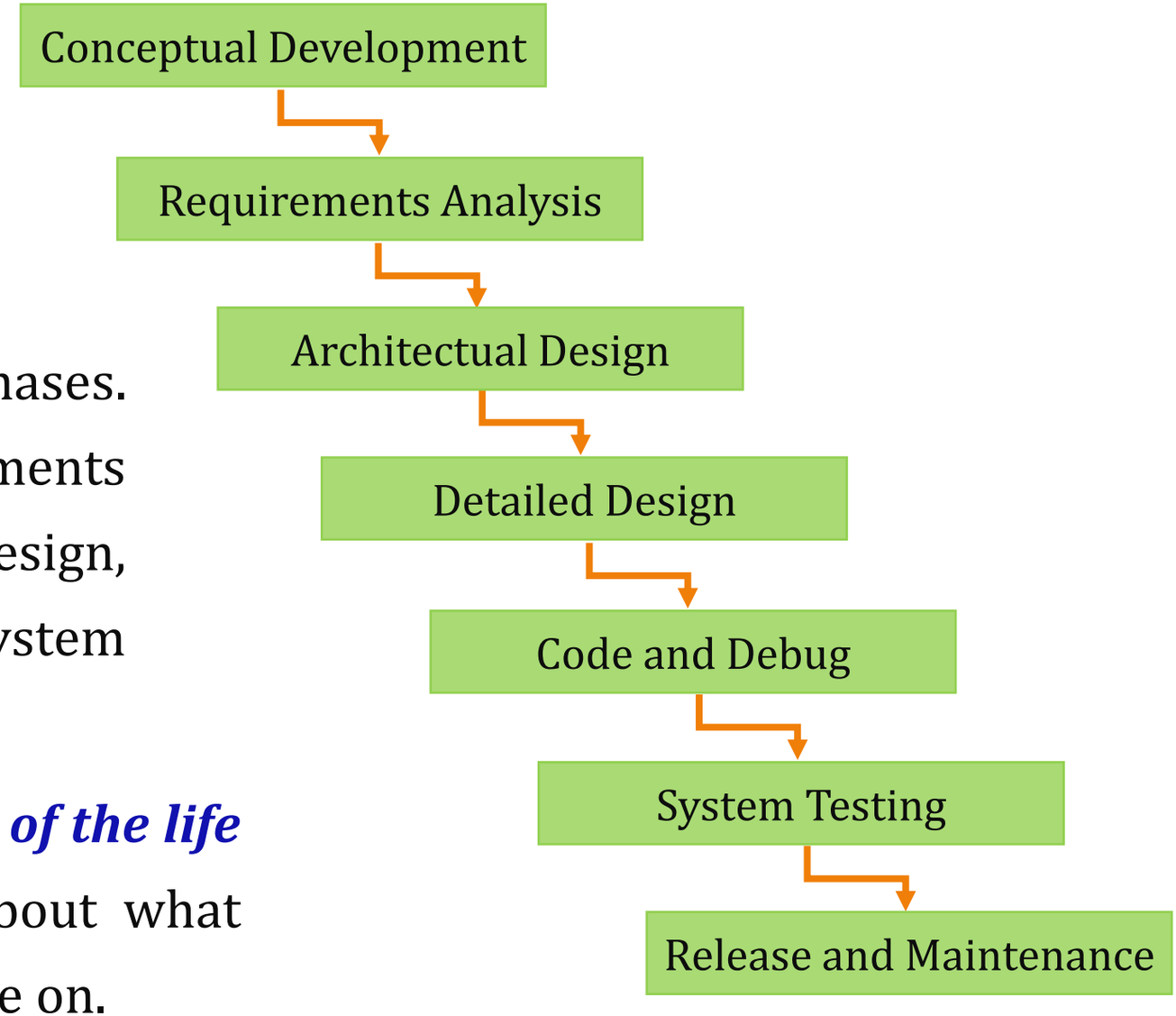
Code and Fix Model

- In this model, there are typically ***no estimates or schedules***, ***no formal requirements***, ***no required documentation***, ***no architectural planning***, ***no quality assurance or formal testing***, and ***no maintenance involved***.
- Therefore, this model ***works well for small, single-person projects***. It is particularly suitable for quick and dirty proofs-of-concept, such as validating architectural decisions, demonstrating a quick version of a user interface design, or understanding of a larger problem you are working on.



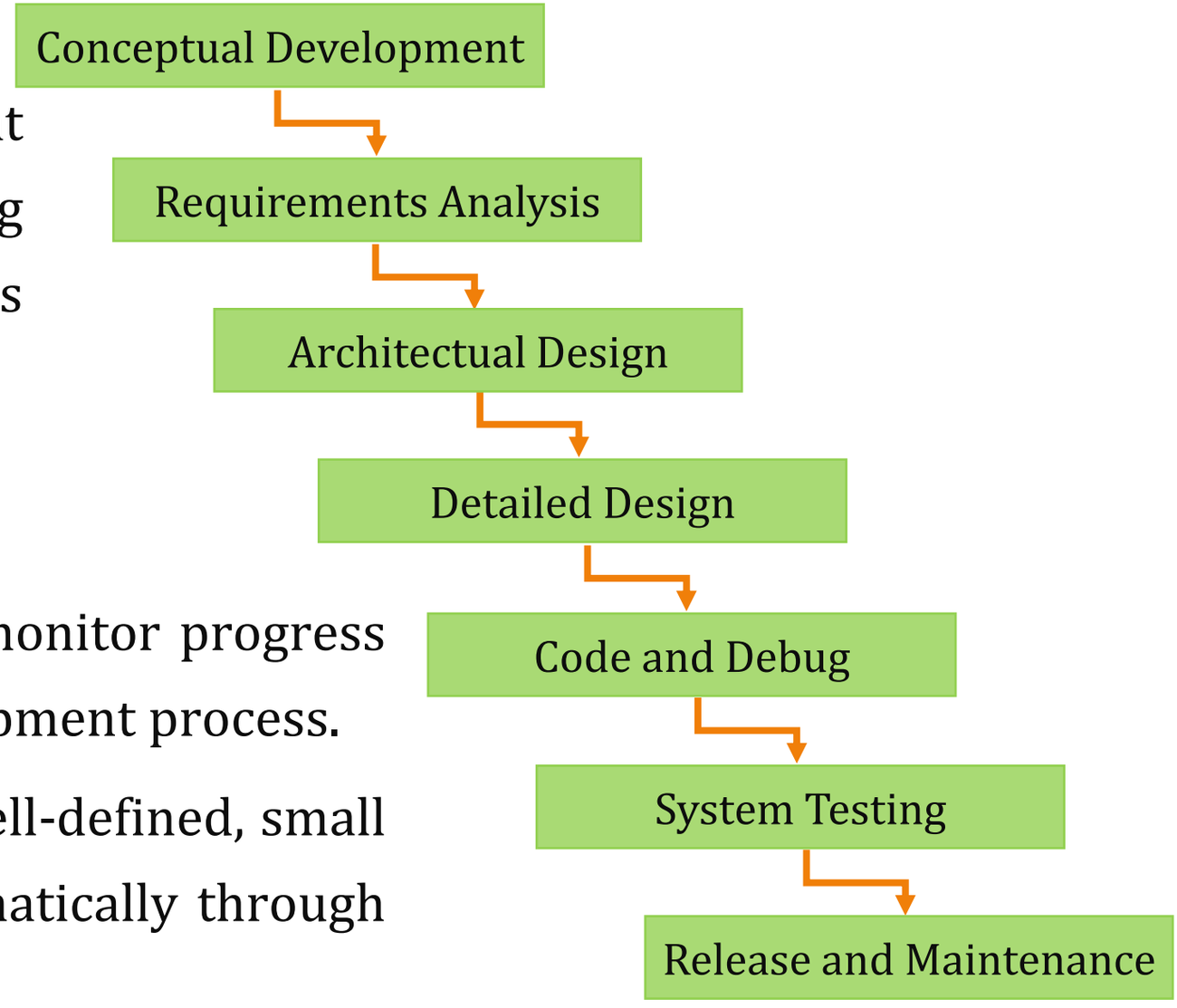
Waterfall Model

- The **Waterfall model** is the first and most traditional of the ***plan-driven process models***.
- It addresses all the standard life cycle phases. It progresses nicely through requirements gathering and analysis, to architectural design, detailed design, coding, debugging, system testing, release, and maintenance.
- This model ***isolates the different phases of the life cycle*** and forces developers to think about what they really need to know before they move on.



Waterfall Model

- It requires ***detailed documentation*** at each stage, along with reviews, archiving of the documents, sign-offs at each process phase, and configuration management.



- It is a good model for
  - ***large projects*** as it helps managers monitor progress and maintain visibility over the development process.
  - ***inexperienced teams*** working on a well-defined, small project because it guides them systematically through the life cycle.

Waterfall Model Problems

- There are two fundamental and related problems with the Waterfall model that make it very difficult to implement effectively:
  - **Problem 1:** This model requires that ***phase* N *must be completed before moving on to phase* N+1.** This means that all requirements must be finalized before starting architectural design, coding and debugging must be finished before start testing, and so on. In theory, this approach is great. With a complete and clearly understood set of requirements, the development team can confidently move on to system design and implementation. However, this rarely happens. There are few projects in which all the requirements are known at the beginning, or where these requirements are correctly understood by all stakeholders, or where big things are not changed during development. Thus, completing one phase before starting the next is problematic.

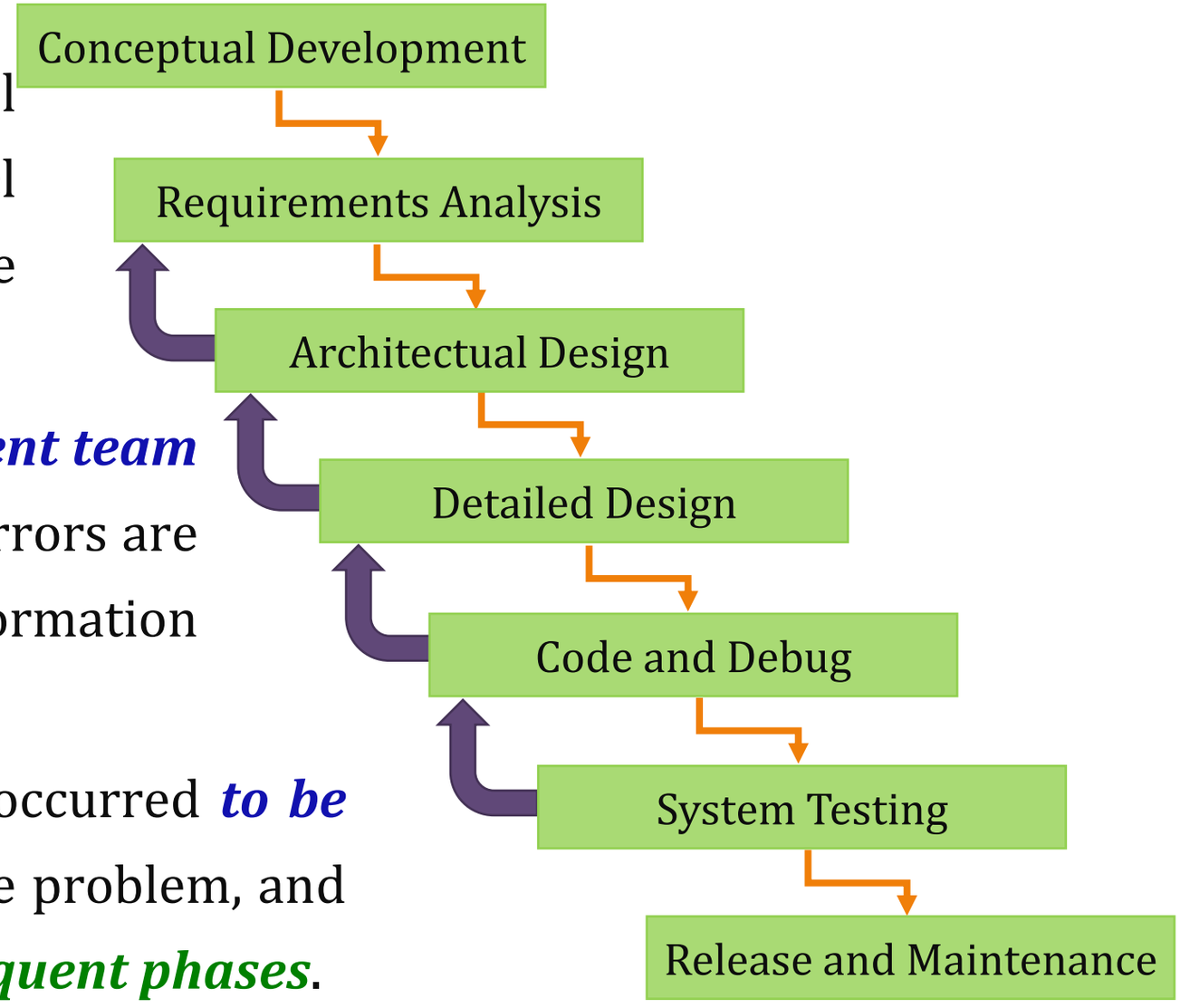
Waterfall Model Problems

- **Problem 2:** Waterfall ***has no provision for backing up to earlier stages for adjustments.*** It is fundamentally based on an assembly-line mentality for developing software. This means that ***no way to go back and rework your design if you find a problem during implementation***. You really have to nail down one phase and review everything in detail before you move on. In practice, the world doesn't work this way. You never know everything you need to know at exactly the time you need to know it.

- Thus, because the waterfall is not a good practical model, it immediately ***morphs into*** a slightly different one called Waterfall with feedback model.

Waterfall With Feedback Model

- In contrast to the traditional waterfall model, the waterfall with feedback model provides feedback paths from each phase to its preceding phases.
- The feedback paths ***allow the development team to go back to previous phase(s)*** when errors are detected at any phase or when a new information about the project is uncovered.
- This enables the phase in which errors occurred ***to be revisited*** to correct errors and rectify the problem, and these changes ***are reflected in the subsequent phases***.



Waterfall With Feedback Model Problems

- This model ***still has the same advantages of the traditional Waterfall model,*** particularly for large projects and inexperienced teams.
- However, the Waterfall with feedback model also has several disadvantages :

  (1) It is ***still quite rigid***.

  (2) It really ***messes with your scheduling big time*** because in any phase there can be unexpected moves back to a previous phase of development.

  (3) It becomes ***harder to determine when the project will be finished***.
- Although ***the waterfall with feedback model recognizes that all the requirements aren't typically known in advance***, it doesn't sufficiently incorporate this reality into the development process. As a result, these limitations led to the evolution of new process models designed to better address the scheduling and uncertainty issues.

Iterative Process Models

- Since all the requirements aren't typically known in advance, the best practice is to ***iterate and deliver incrementally***. Each iteration is treated ***as a self-contained "mini-project",*** including requirements analysis, design, coding, testing, and internal delivery. At the end of each iteration, a ***fully-tested and integrated version of the system is delivered to*** internal stakeholders. Their feedback is then collected and incorporated into the planning of the next iteration.
- In iterative process models, the development team begins with the initial requirements that are ***clearly defined*** and ***prioritizes them***, typically based on the ***customer's ranking of which features are most important to deliver first***.
- The team then selects ***the highest-priority requirements*** and plans a series of iterations, each representing a complete development cycle.

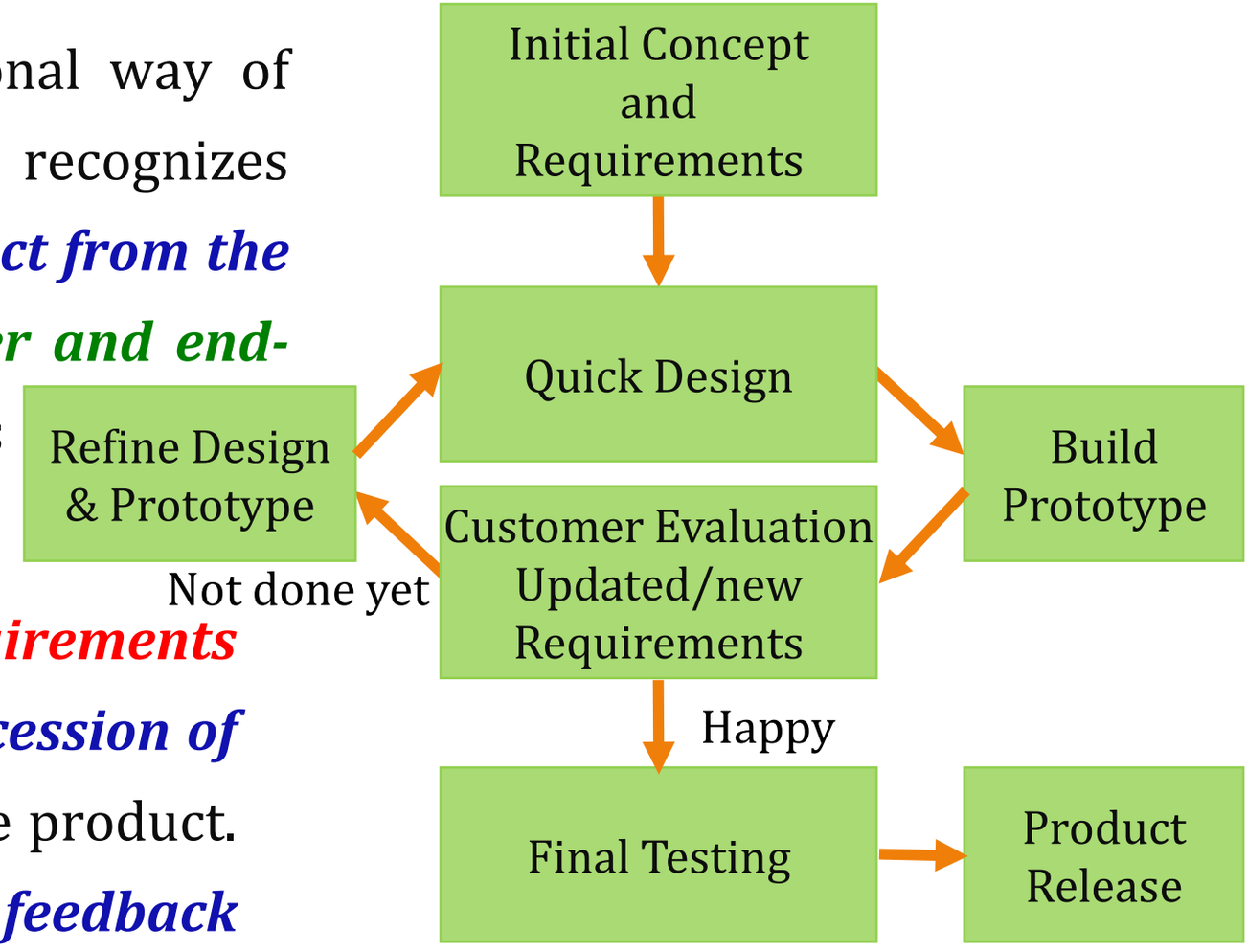
Iterative Process Models

- In each iteration, the team ***selects the next set of highest priority requirements*** (including some that have been discovered during the previous iteration) ***and repeat this process until the final system is completed***.
- At the end of each iteration, the team produces ***a complete, working, and robust product***, but with fewer features than the final product will have.
- Iterative processes **follow a fundamental principle**:

  The project has a binary deliverable. This means that, on the scheduled completion date of an iteration, a system with the objective completion criteria is either delivered and accepted by the customer, or it is not.
- The key to ***iterative software development*** is to ***analyze some***, ***design some***, ***code some***, and ***test some*** every day.

Evolutionary Prototyping Model

- Evolutionary prototyping is the traditional way of implementing the incremental model. It recognizes that it is ***very hard to plan the full project from the start*** and that ***the feedback of customer and end-user is a critical*** element of good analysis and design.
- In this model, the team ***prioritizes requirements*** as they are received and ***produces a succession of increasingly feature-rich versions*** of the product. Each version is ***refined using customer feedback*** and the results of integration and system testing.



Evolutionary Prototyping Model

- Evolutionary prototyping model **advantages**:
  - It provides ***improved progress visibility*** for both the customer and project management.
  - It provides ***good customer and end-user input directly influences product requirements*** and helps in prioritizing those requirements effectively.

Evolutionary Prototyping Model

- Evolutionary prototyping model **disadvantages**:
  - It may lead to ***unrealistic schedules***, ***budget overruns***, and ***overly optimistic expectations regarding progress.*** These often occur because implementing a limited set of requirements in a prototype can give the impression of real progress for a small amount of work. Conversely, putting too many requirements in a prototype may lead to schedule delays, because complexity increases and estimation becomes inaccurate.
  - It may result in ***bad design*** because the system evolves over time as the requirements change.
  - It may result in ***low maintainability*** because the design and code evolve as requirements change. This may cause excessive rework, schedule overruns, and increased difficulty in fixing bugs after release.

Evolutionary Prototyping Model

- Evolutionary prototyping works best with ***small, experienced teams*** who have worked on multiple projects. Such cohesive teams are productive and adaptable, able to focus on each iteration and usually produce coherent, extensible designs that a series of prototypes requires. This model is generally not recommended for ***inexperienced teams***.
- It is particularly well-suited for projects with ***changing or ambiguous requirements***, as well as for ***applications*** in ***poorly understood domains***.
- This is the model that evolved into the modern **agile development processes**.


Thank
you